

(=Referer)

3. リクエストメッセージ (POST など) 内へこの値を添付し、変更はしない。信用度は高い。
+ 秘密情報 (セッション ID など) を URL に含めず、2. と一致する URL

→ この値を送信する場合、GET は OK、POST など送る → hidden (リクエストに送らば)

セッション管理には Cookie が有用。

↑ セッション ID のみを格納しておくのが一般的

ブラウザ上では不可能なため、この部分に脆弱性があるが、書き換えられず (第三者が覗く)

→ 連続して生成、認証時にセッション ID を更新して使い回す

同一オリジンポリシー ... JS などから、サイトをまたがってアクセスを禁止するセキュリティ上の制限
URL のホスト名やポート番号、プロトコルが一致する必要がある

CORS ... 異なるオリジンとのデータのやり取りを可能にする仕様

被アクセス側が許可するように設定する (Access-Control-Allow-Origin)

シンプルにリクエストではない場合、プリフライトリクエストが必要になる場合がある

認証情報の取り扱いについては、withCredentials で制御する

4. 脆弱性には、処理に起因するものと入力に起因するものがある

→ データの移譲を示すマークを埋めさせ、その後 (コメント) と添付する

入力処理 1. 文字 encoding の妥当性検証

2. " の変換

→ 入力メッセージの仕様は厳格に依存する

3. 数値の妥当性検証

→ 必ずしも必要ではない

ヌルバイト攻撃 ... 値ゼロのバイトが言語の仕様上、文字の移譲だと扱えらるため、これを突いた攻撃

→ 制御文字を受けつけない、最大文字数を定める、数値の上限と下限を定める、データ形式のチェック

→ 正規表現は有用 (ex. POSIX)

XSS ... 動的な箇所を発生し、外部から入力可能なページに存在する脆弱性が原因

→ クッキー値の盗み出し、その他の JS による攻撃、画面の書き換え

・ 「<」、「>」、「&」、「"」、「'」などの記号に適切なエスケープ処理を施すことが重要 + HTTP リスポンス (文字エンコーディング) の明示

・ エラーも画面に表示させないようにしないといけない

SQL インジェクション ... SQL 呼び出し時に意図しない操作が行われる (不正な SQL が実行される)

エラーメッセージに情報を表示させない、SQL のロギングを遮断して不正な操作をさせる

→ プレースホルダを用いて SQL 文を構成するのが安全 (特に静的プレースホルダ)

→ 複文の禁止や、例外処理を適切に設定することも有用

→ 偽造

CSRF ... リクエストを偽造すること、重要な処理を不正に実行させる攻撃

Cookie は親サイトを理由してリクエストに自動で割り当てられることがある。

→ トークンの埋め込み (input type=hidden) や Referer チェックが効果的

→ フォームオール等、Referer を抑止し、おと断する

クロスサイトスクリプティング ... iframe と CSS を巧みに駆使して、想定外のユーザー操作を実行させる攻撃

→ X-Frame-Options を設定すれば、十分に対応できる (同一ポリシーで許可)

セッションハイジャック ... セッション ID の漏洩により、悪用されたりする攻撃

→ 推測できない ID を生成、ID を URL に埋め込まない、認証時にセッション ID を更新

→ 言語やフレームワークのセッション管理機構を使いこなす

Cookie にのみ保存する方が安全

オープンリダイレクト... URL等にリダイレクト先が記載されており、一定処理後にそこにリダイレクトすること
→ リダイレクト先の任意性もなく、ドメイン名をフェイクする などの対策

HTTPヘッダインジェクション... HTTPレスポンスヘッダを改変することによる攻撃
別ドメインにリダイレクトする1つの場合
7つのページを等しいと見做す
レスポンスヘッダの改変
レスポンスヘッダの偽装
他にも、CRLFインジェクションやHTTPレスポンス分割攻撃などがある
ヘッダ情報の区切りが改行になっちゃうと脆弱性の根源
⇒ 外部からのリクエストでレスポンスヘッダを改変できない
専用APIに依存
改変できちゃうのはおかしな話

Cookieの脆弱性... 本方法に問題がある場合、Cookieに保存するのは正しい情報か格納できているか
セキュリティ属性不備

→ HTTPSの場合のみ、セッション情報をブラウザからサーバーに送信する
→ セキュリティ属性をつける、トーンを用いて代用する など
上記以外にも Domain (複数サーバー共有), Path (パスに限定するセッション)
Expires (期限), HTTPOnly (JSから操作不可) 属性が有用

XMLヘッダインジェクション... XMLのテキスト形式は、空白を使用しない判別する形式になっている
階層の使い分け次第で、XMLの内容をいくらでも改変できる
⇒ 専用のライブラリを使用する、階層コードフェイク、外部からのリクエストでXMLリクエスト

ファイルトラバース脆弱性... サーバ上のファイル名を指定できるアトリビュート、ファイル名をフェイクして、サーバ上の
データの閲覧、削除、改変、サーバのリポートの実行などができちゃう
⇒ 外部からファイル名を指定できないようにする
ファイル名をフェイクして、リクエストに指定できないようにする
組み立てたファイル名のアトリビュートでフェイクする
これはOK
これはOK

OSコマンドインジェクション... シェル経由でのOSコマンド呼び出しを悪用することによる脆弱性
シェル呼び出しを行う関数等を使用したり、特定のポートのポート番号をエスケープして呼び出す
⇒ できるだけシェル操作を避け、止むを得ない場合にエスケープ処理を要する2重使用
アプリケーションユーザの権限を最小限にする
英数字限定

DOS攻撃... 大量のデータを送信することにより、サーバに負荷をかける攻撃
アプリケーションに対して、ファイル容量や、幅、高さ、色数をフェイクしておくと良い
⇒ ファイル使用後のサイズを削除する
障害は少ない
test.php.png など

サーバ側リポート実行... サーバ側でリポートファイルを実行するプログラム、その後実行することによって任意の操作を行う
(プログラムでファイル名を指定) ⇒ プログラムで指定したファイル名をリポートに置き換える、プログラム時に拡張子をフェイクする
⇒ リポート経由で、ファイル名をプログラムで指定する(プログラムで指定する)

クロスサイトリクエストフォージェリ... クロスサイトリクエストフォージェリは即時実行されるタイプのXSS (Content-Typeを認識する必要)
(リクエストのタイプを指定) ⇒ Content-Typeを正しく設定、X-Content-Type-Options: nosniff を設定
+ Content-Disposition
⇒ Content-Typeのみを解釈するようにする

PDFの仕様によるバグ... ブラウザ上でPDFを開いた場合に、セッション情報が罠サイトに飛ばされちゃう (バグ)
⇒ PDFはブラウザ内で開くようにするのは、アプリケーションで指定する

リモートファイルインクルード攻撃... ファイルインクルード部分は任意のリクエストを渡すことのできる場合に起こる攻撃
⇒ 仕様を変更、インクルードファイルの妥当性をフェイク、allow-url-include = off (設定)

evalインジェクション... evalや、内部的に使う使用している関数に外部からのリクエストを渡すことにより任意のリポートを実行できる
⇒ そもそもevalを使用しない、外部からのリクエストをevalに渡さない、渡す場合は妥当性をフェイクする
⇒ 3つのライブラリなどは別の方法がある

JSONプリパースを行う際は、JSON形式の安全な、セッションIDなどの書き換えできない
形式でデータの受け渡しをするのが安全

XML外部参照実行... XMLが外部ファイルを読み込むことを可能にするという機能をこのように攻撃 (XFE)

⇒ JSON形式でやり取りする。XMLのEntity Loaderを禁止する

→ 別問題

競合状態の脆弱性... 共有資源 (共有メモリ、キャッシュ等) の排他制御が不十分の場合に競合が起こる

⇒ 必ずだけ共有資源 (インテリジェント変数等) を使用せず、排他制御をかける

⇒ 必ずだけ
CORS対応
XMLHttpRequest

キャッシュサーバ (リバースプロキシ) でも同様 ⇒ レスポンスを制御する

→ 必ずだけレスポンスはDOS脆弱性

JSON... JavaScriptのオブジェクトリテラルを模した、データ交換形式 (表現がやや冗長なXMLの代替)

JSONP... CORSが導入前に、異質なオリジンからデータを取得する方法として使用されたJSON
JSONデータを引数とするコールバック関数呼び出しの形式で送る (script要素の動的取得)

JSONエスケープの不備... JSONが別ファイルから読み込まれる際、エンコードミスが生じる脆弱性

⇒ エンコードは、信頼できるライブラリを使用する

JSONでのXSS... JSONを直接閲覧する際、JSONエンコードしたスクリプトが埋め込まれることがある

⇒ MIMEタイプを正しく設定し、UNICODEエスケープを行う

WebAPIのCSRF... GETリクエストでデータの操作ができて、不適切なMIMEタイプ、XMLHttpRequestの悪用による攻撃

⇒ CSRFトークンのセッション変数の管理、二重送信キー、カスタムリクエストヘッダがある

→ トークンとHTTPリクエストヘッダの二重送信

異なるオリジンでは、リクエストが渡り合わない

JSONNTags... script要素の中で無理矢理JSONファイルの中身を開く

DOM Based XSS... JavaScriptでの処理の不備により、XSS脆弱性が発生してしまう

⇒ 外部からの入力に対しての適切な状態を作らない

location.hrefやinnerHTML, eval, jQueryの動的生成等

上記以外にもpostMessageによる脆弱性や、オプティミズドインポート脆弱性もある

5. ログイン機能に対する攻撃

- SQLインジェクション
- ブルートフォース攻撃 (ユーザーIDやパスワードの暴力攻撃)
- リレーショナルエンジニアリング (SQLインジェクション)
- フィッシング

→ ユーザーIDの防止は効果的
→ ユーザーIDとパスワードの両方を守る

⇒ 強固なパスワードが重要で、厳しい条件を与えることが大事

アカウントロックを取り入れるのも効果的 (UXの観点も考慮が必要)

パスワード認証に対する攻撃

- 辞書攻撃... 使用頻度の高いパスワード群を順に試す
- ジョイントアカウント探索
- リバーブブルートフォース攻撃... パスワード固定で、ユーザーIDを変える
- パスワードズプレ攻撃... IDもパスワードも固定で、少量のパスワードでIDをID
- パスワードリスト攻撃... 他サービスからのパスワードリストを試す

⇒ 二段階認証、パスワード変更 (作成時)、ログイン失敗率の判定

→ 通常の利用と違う場合や、初回ログインはなし

パスワード保護には、XSSやSQLインジェクションを呼び出すのを防ぐ必要がある

→ 基本的には複合的な

任意長のデータを固定長のデータ (ハッシュ) に圧縮する関数 (ハッシュ関数) を用いてハッシュ化する (暗号学的ハッシュ関数)

これに抵抗するように、

オフラインブルートフォース攻撃... ハッシュ値からの復号を目指すブルートフォース

レインボータブル... ハッシュ値の導引表を用いる

⇒ ユニコード (ハッシュの元データは文字列で追加) やスループット (計算速度遅延) が効果的

4) 9 ~

→ セッションIDはここ

自動ログインは認証状態が長く続くため、XSSやCSRFなどのリスクが高まる ⇒ トークンを利用する方が安全

ログインフォームは、
パスワード入力欄はマスク表示 (最近は切替式が主流) ユーザー入力欄 読み上げソフト対策
HTTPS を使用 (盗聴の防止と悪サイトの検知)

エラーメッセージには、攻撃者のヒントになる情報を含めずにはいけない (ただし、ログインのURLは隠す必要あり。一長一短)

ログインはPOSTメソッドで行う

ユーザー登録では、
→ ユーザーIDの受信確認 トークンとURLから、トークンの送信に失敗したら
→ ユーザーIDの重複防止
→ ユーザーの自動登録への対策 CAPTCHAやreCAPTCHA, IPアドレス認証などがある
→ 大量にユーザーを作成できず、迷惑メール送信の送信元アドレスとして利用できなくなるへの対策

パスワード変更では、現在のパスワードも確認すること。セッションIDの変更も必要。またパスワードも新しいものに設定する。

パスワードリセットは、仮パスワードを作成する。その後トークンを送信する方式が望ましい。

認可処理の不備には、
→ ユーザーのURLを知っているだけで、不正にアクセスできる
→ IDを管理するだけで、権限外の情報にアクセスできる Hidden Input Cookie
→ ユーザーの表示・非表示の可否を制御できず、不正な書き換えで権限外にアクセスできる
⇒ 認可制御の要件定義、ルール設定

→ 英語のみ

→ 多言語対応

6. 文字集合 ... 文字とその識別番号の振り分けを定義したもの。ASCIIやUnicodeなどがある

異なる文字集合間の変換は、割り当て数字が同じ記号でも異なる場合があることに注意

文字エンコーディング ... 文字集合の符号化に関する仕組み

→ 5C問題

Shift-JIS ... 多重割り当てが起きている箇所があり、扱いづらい (XSS脆弱性にもなる)

UTF-8 ... 5C問題は発生せず、比較的的安全に使える。非最短形式の問題がある